`radcoolpv`
Physics, Numerical Models, YAML Cases, and Validation

Project manual

30 July 2026

Contents

1 Purpose and scientific status

`radcoolpv` calculates optical properties of periodic photonic structures for radiative cooling and couples them to a silicon PV thermal/electrical model. The active implementation is based on the MATLAB+Lua/S4 `matlab-radCoolPV` model. Live optics use only the Stanford S4 Python extension. S4 is imported lazily; reduced-spectrum and free-form readers can run without it.

The package reports spectral reflectance R, transmittance T, total absorptance A, silicon-layer absorptance A_{Si}, and, under the conditions stated in Section 4.4, emittance. The thermal stage reports operating temperature, I–V characteristics, maximum-power-point output, efficiency, and a temperature reduction relative to a user-specified reference.

The library is not a universal PV module model. Its central assumptions are a periodic coherent optical unit cell, scalar material permittivities, a lumped steady-state thermal balance, and the silicon diode model inherited from the MATLAB code. Validation status is evidence-labelled in Section 9; a passing software test is not treated as proof of physical validity.

2 Installation

```
./install.sh
source .venv/bin/activate
python -c "import radcoolpv; print(radcoolpv.__version__)"
```

The thermal/PV stage and spectrum readers use the Python dependencies installed by `install.sh`. Live optics requires a separately compiled S4 Python module:

```
brew install fftw suite-sparse openblas lapack boost
git clone https://github.com/phoebe-p/S4
cd S4
make -f Makefile.m1 S4_pyext
python -c "import S4; print(S4.__file__)"
```

The S4 import occurs only when a case with `geometry.source: s4` executes.

On Windows, use WSL2: the Python package is platform-independent, but S4 is a compiled extension with no wheel and a Unix makefile build, so inside WSL every command above applies unchanged. Native Windows still runs the thermal/PV stage, the free-form reader, and any case resuming a stored spectrum — which covers Validations A and B, the Validation C thermal cases, and step 2 of A.1 — and a case that needs live optics fails with an explicit message rather than a wrong answer. Note that `install.sh` is a shell script; on native Windows install with `pip install -r requirements.txt` followed by `pip install -e .` instead. Git is optional everywhere: without it the manifest records a null commit and the run proceeds.

3 Where each quantity is computed

Every equation in this manual is implemented in one place. The table below is the index from physics to source, so a reader checking a result never has to guess which function produced it.

Quantity	Module	Function
YAML parsing, defaults, validation	`config.py`	`from_dict`, `validate`
Stage sequencing	`pipeline.py`	`run`
Layer stack from geometry	`optics/geometry.py`	`build_structure`, `_photonic_layers`
Permittivity per material	`materials/tabulated.py`, `materials/analytic.py`	`_interpolator`, `drude_si3n4`
R, T, A, A_{Si} from S4	`optics/s4_backend.py`	`sweep`, `_fluxes`
Angular quadrature nodes	`config.py` (`SimulationConfig`)	`directions`
Directional $\rightarrow$ hemispherical	`optics/directional.py`	`reduce`, `_selected`
Resuming a stored spectrum	`optics/directional.py`	`from_reduced_file`
Band averages	`optics/averages.py`	`band_average`, `pv_band_averages`
Solar spectrum, photon flux	`thermal/spectra.py`	`load_solar`, `load_atmosphere`
Planck-weighted radiated power	`thermal/radiative.py`	`rad_power`
Band gap, J_{sc}, J_0, Auger, I–V	`thermal/pv.py`	`solve_iv`
Luminescence term	`thermal/pv.py`	`non_thermal_power`
MPP and V_{oc} refinement	`thermal/pv.py`	`refine_peak`, `open_circuit_voltage`
Energy balance, T_{eq}	`thermal/energy_balance.py`	`run`, `_zero_crossing`
CSV, export, manifest	`io/clean_writers.py`	`_write_csv`, `write_optics_export`, `write_run_json`

4 Optical model

4.1 The S4 problem

The YAML structure is ordered from the incident medium toward one semi-infinite terminal layer. S4 expands the in-plane fields in a Fourier basis of size `simulation.s4_modes`. Flat, circular, rectangular, and polygonal regions are supplied to S4 analytically; spheres and hemispheres use the configured stack of circular slices.

At each wavelength, direction, and polarization, the code evaluates the integrated normal Poynting flux. With incident flux P_{inc},

$$R = -\frac{P^{-}_{\mathrm{top}}}{P_{\mathrm{inc}}}, \tag{1}$$

$$T = \frac{P^{+}_{\mathrm{bottom}}}{P_{\mathrm{inc}}}, \tag{2}$$

$$A = 1 - R - T. \tag{3}$$

Unlike the old Lua output, T is normalized by incident flux at oblique incidence. The silicon-layer absorptance uses the net flux at the two interfaces of the silicon layer itself:

$$A_{\mathrm{Si}} = \frac{P_{\mathrm{net}}(z_{\mathrm{Si,top}}) - P_{\mathrm{net}}(z_{\mathrm{Si,bottom}})}{P_{\mathrm{inc}}}. \tag{4}$$

This prevents absorption in a layer below silicon from being assigned to the PV cell.

Because A is formed as $1 - R - T$, energy conservation is structural rather than a diagnostic: what it tests is the flux normalisation, not the solver. The residual $|R + T + A - 1|$ computed in memory is at machine precision ($\sim 10^{-16}$); the $\sim 10^{-7}$ seen in stored spectra is the `%.6e` text format, not a physical error.

4.2 Unit cell and shape discretisation

`geometry.lattice.type` selects the Bravais cell. `square` builds vectors $(x, 0), (0, x)$ and *ignores* `lattice.y`. `hexagonal` builds $(x, 0), (0, y)$ and additionally places a second copy of every circular motif at the cell centre $(x/2, y/2)$: the hexagonal array is represented as a centred-rectangular supercell with two motifs. For nearest-neighbour pitch p, the correct entries are

$$x = \sqrt{3}\, p, \qquad y = p, \tag{5}$$

which places all nearest neighbours at distance p. The centred motif is added only for `cylinder`, `sphere`, and `semisphere`; for `triangle` and `grating` a hexagonal setting changes the cell vectors but yields a single motif, so those shapes are effectively rectangular.

Curved shapes are stacks of uniform slices, since S4 layers are z-invariant. For a sphere or hemisphere of radius a discretised into $N =$ `discretization_layers` slices of thickness $\delta = 2a/N$, slice i is a circle of radius

$$r_i = \sqrt{\max(a^2 - (y_i - a)^2,\ 0)}, \qquad y_i = 2a - \tfrac{\delta}{2} - i\,\delta. \tag{6}$$

A hemisphere keeps the top $\lfloor N/2 \rfloor$ whole slices and, for odd N, one half-thickness slice straddling the equator, so the dome spans exactly a for either parity. A triangle of base b and height H uses rectangles of half-width $b\,y/2H$. `grating` is a one-dimensional lamellar layer: ridges of `photonic_material` with period `lattice.x`, duty r, and a vacuum groove of half-width $(1 - r)\,p/2$ spanning the full cell in y.

N is a convergence parameter, not a free choice: a slice stack is a staircase approximation of a curved surface, and only `cylinder`, `grating`, and `flat` are represented exactly.

4.3 Directions and polarization

One YAML case supports one of three modes:

- `normal`: $\theta = 0$;
- `specific`: one user-defined polar angle θ and azimuth ϕ;
- `hemispherical`: a two-dimensional (θ, ϕ) quadrature.

`polarization` is `TE`, `TM`, or `unpolarized`. TE and TM are never assumed equal solely because $\theta = 0$; anisotropic unit cells such as one-dimensional gratings can give different normal-incidence responses. Unpolarized output is

$$X_{\mathrm{unpol}} = \frac{X_{\mathrm{TE}} + X_{\mathrm{TM}}}{2}, \qquad X \in \{R, T, A, A_{\mathrm{Si}}\}. \tag{7}$$

For one polarization p, the hemispherical property is

$$X_{\mathrm{h},p}(\lambda) = \frac{1}{\pi} \int_0^{2\pi} \int_0^{\pi/2} X_p(\lambda, \theta, \phi) \cos\theta \sin\theta \, d\theta \, d\phi. \tag{8}$$

The implementation applies Gauss–Legendre quadrature in $u = \sin^2\theta$ and a uniform periodic azimuth rule. It avoids the grazing endpoint and includes a separate zero-weight normal probe for the PV luminescence term. Quadrature weights sum to one. A true hemispherical result therefore requires convergence with respect to both `hemisphere_theta_points` and `hemisphere_azimuth_points`.

4.4 Emittance and Kirchhoff's law

The code labels total absorptance as emittance using directional Kirchhoff reciprocity,

$$\epsilon_p(\lambda, \theta, \phi, T) = A_p(\lambda, \theta, \phi, T). \tag{9}$$

This identification requires a passive reciprocal structure in local thermal equilibrium, evaluated with the same wavelength, direction, polarization, and temperature-dependent material state. It is not automatically valid for nonreciprocal, active, nonlinear, or nonequilibrium emitters. The present material models are temperature-independent.

4.5 Materials and numerical validity

Tabulated models interpolate n and k linearly and use $\epsilon = (n + ik)^2$. Requests outside a table's wavelength interval raise an error; endpoint clamping and silent extrapolation are forbidden. Validation E uses the unmodified refractiveindex.info Olmon evaporated-Au record. Its SiO2 and Si refractive-index tables are inherited from `matlab-radCoolPV`: the former is labelled Palik/Kitamura; the latter has no bibliographic source and is made lossless to implement the paper's explicit nonabsorbing-Si assumption. The current refractiveindex.info database has no records labelled with the paper-cited Palik/Kitamura Si or SiO2 sources. This provenance gap is retained as a limitation rather than silently replacing the models.

Every scientific result must demonstrate:

1. passivity and finite R, T, A, A_{Si}, with $R + T + A \approx 1$;
2. convergence in S4 Fourier basis size;
3. for hemispherical results, convergence in both angular counts.

5 Atmosphere, thermal balance, and PV model

5.1 Atmospheric and thermal radiation

For zenith transmittance $\tau_{\mathrm{atm}}(\lambda)$, the directional atmospheric emittance is

$$\epsilon_{\mathrm{atm}}(\lambda, \theta) = 1 - \tau_{\mathrm{atm}}(\lambda)^{1/\cos\theta}. \tag{10}$$

The surface–atmosphere product is integrated with the same angular quadrature as the surface emittance. With Planck spectral radiance $B_\lambda(T)$,

$$P_{\mathrm{rad}}(T) = \pi \int \epsilon_{\mathrm{h}}(\lambda) B_\lambda(T)\, d\lambda, \tag{11}$$

$$P_{\mathrm{atm}}(T_{\mathrm{amb}}) = \pi \int \langle \epsilon \epsilon_{\mathrm{atm}} \rangle_{\mathrm{h}}\, B_\lambda(T_{\mathrm{amb}})\, d\lambda, \tag{12}$$

$$P_{\mathrm{conv}}(T) = h\,(T - T_{\mathrm{amb}})\,. \tag{13}$$

The convection coefficient h is the total effective nonradiative coefficient for the reference area used by the balance. It is an external lumped-model input and is not deduced from geometry or weather data. A per-surface coefficient must be combined explicitly by the user; the library does not multiply h by an assumed number of exposed surfaces.

`rad_power` returns the radiance integral $\int \epsilon(\lambda) B_\lambda(T)\,\mathrm{d}\lambda$ in $\mathrm{W\,m^{-2}\,sr^{-1}}$, with

$$B_\lambda(T) = \frac{2hc^2}{\lambda^5} \left[\exp\!\left(\frac{hc}{\lambda k_B T} \right) - 1 \right]^{-1}, \tag{14}$$

evaluated on the micrometre grid. The caller multiplies by π to integrate the cosine-weighted hemisphere, so $\pi \int B_\lambda\,\mathrm{d}\lambda$ over a broad band reproduces σT^4; that identity is a useful check on any new wavelength range. Because the integral is trapezoidal on the configured grid, a range that truncates the Planck peak silently understates P_{rad}: at 300 K the peak lies near 9.7 μm and a 4–30 μm window is the usual minimum.

5.2 Solar absorption and steady state

Total absorbed solar power is

$$P_{\mathrm{sun}} = \int A(\lambda) I_{\mathrm{AM1.5G}}(\lambda)\, d\lambda. \tag{15}$$

For a literature cooling curve that prescribes absorbed solar power directly, the `absorbed_solar_power` key in the `thermal` section overrides this integral. Validation E uses the paper value 808 $\mathrm{W\,m^{-2}}$.

The cooling-power convention is

$$P_{\mathrm{cool}}(T) = P_{\mathrm{rad}} - P_{\mathrm{atm}} + P_{\mathrm{conv}} - P_{\mathrm{sun}} + P_{\mathrm{MPP}} + P_{\mathrm{nt}}, \tag{16}$$

where P_{MPP} is electrical power removed from the cell and P_{nt} is the non-thermal luminescence term. Equilibrium satisfies $P_{\mathrm{cool}}(T_{\mathrm{eq}}) = 0$.

Cooling-curve cases specify their temperature interval in YAML. Standard PV cases evaluate T_{amb} through $T_{\mathrm{amb}} + 150$ K. If the balance does not bracket a root, the run raises an error. It never reports a sweep endpoint as an equilibrium temperature.

5.3 I–V calculation

The short-circuit current uses IQE, silicon-layer absorptance, and the solar photon flux up to the silicon bandgap:

$$J_{\mathrm{sc}} = q \int_0^{\lambda_g} \mathrm{IQE}(\lambda) A_{\mathrm{Si}}(\lambda) \Phi_{\mathrm{sun}}(\lambda)\, d\lambda. \tag{17}$$

At each T, V, the current density J solves

$$0 = -J + \frac{V - JR_s}{R_{\mathrm{sh}}} + J_0(T)\left[\exp\left(\frac{q(V - JR_s)}{k_B T}\right) - 1\right] + J_{\mathrm{Auger}}(T, V) - J_{\mathrm{sc}}. \quad (18)$$

Output power is $-JV$. Each voltage is solved with `scipy.optimize.fsolve`, warm-started from the previous voltage's solution, which keeps the strongly exponential diode equation in its convergence basin across the sweep.

The saturation current is itself a spectral integral rather than a fitted constant, using the same IQE and silicon absorptance against a blackbody photon flux at the cell temperature:

$$J_0(T) = q \int_0^{\lambda_g} \mathrm{IQE}(\lambda) A_{\mathrm{Si}}(\lambda) \Phi_{\mathrm{bb}}(\lambda, T)\, \mathrm{d}\lambda. \quad (19)$$

Auger recombination uses the intrinsic carrier concentration

$$n_i = 5.29 \times 10^{19} \left(\frac{T}{300}\right)^{2.54} \exp\left(\frac{-6726}{T}\right) \ \mathrm{cm}^{-3}, \quad (20)$$

with a coefficient $A(T)$ interpolated from five tabulated points (195–372 K, 3.03–4.55×10^{-31} cm^6/s), giving

$$J_{\mathrm{Auger}} = 2qA(T) n_i^3 d_{\mathrm{Si}} \exp\left(\frac{3qV}{2k_B T}\right). \quad (21)$$

Extrapolating outside that temperature range flattens to the endpoint values, so results far outside 195–372 K should be treated as unsupported.

The band-gap wavelength preserves the MATLAB scalar-reduction convention: the original evaluated $E_{g0} - \alpha T^2/(T + \beta)$ with matrix right division on a row vector, which collapses to the single scalar $\alpha\,(T^2 \cdot (T+\beta))/((T+\beta)\cdot(T+\beta))$. One λ_g therefore serves the whole temperature sweep. This is a model-compatibility choice, not a temperature-resolved band-gap model, and it sets the upper limit of every spectral integral above.

5.4 Solving for the operating point

P_{MPP} and the luminescence term depend on V_{mpp}, which depends on the temperature that the balance is trying to find. The code resolves this by fixed-point iteration from $V_{\mathrm{mpp}} = 0.65$ V: assemble the balance, locate T_{eq}, re-evaluate the power curve there, and refine V_{mpp}, stopping once successive values agree to the configured tolerance. Failure to settle raises a `RuntimeWarning` naming the last value rather than silently returning it.

Three numerical choices matter for reproducibility, and all three depart deliberately from the MATLAB original:

- **Equilibrium temperature** is the linearly interpolated zero crossing of P_{cool}, not the nearest grid point. A balance that never crosses zero, or is already non-negative at the lowest swept temperature, raises an error naming the swept interval. An endpoint is never reported as a solution.
- **MPP and** V_{mpp} come from a three-point parabolic fit through the largest sample and its neighbours, so they are not quantised to the `thermal.voltage` step. It falls back to the raw sample at a boundary or for collinear points.

- V_{oc} is the interpolated crossing of $-J$. The MATLAB `find` returned the last grid point before the crossing, biasing V_{oc} low by up to one step (7 mV on the default sweep).

Every equilibrium quantity is then reported at that same interpolated temperature, so T_{eq}, P_{MPP}, V_{oc}, and the fill factor describe one consistent operating point.

5.5 Temperature reduction

When `thermal.reference_temperature` is supplied,

$$\Delta T_{\mathrm{reduction}} = T_{\mathrm{reference}} - T_{\mathrm{eq}}. \tag{22}$$

The user must define the reference physically, for example the operating temperature of a bare module under the same boundary conditions. The library does not invent or silently substitute a reference.

6 Numerical methods and their failure modes

Step	Method	Fails when
Polar quadrature	Gauss–Legendre in $u = \sin^2\theta$	too few nodes for a sharply directional emitter
Azimuth quadrature	uniform periodic rule	anisotropic cells with few azimuth points
Material n, k	linear interpolation, extrapolation refused	requested λ outside the table (raises)
Spectral integrals	trapezoidal on the configured grid	grid truncates the Planck peak or undersamples the solar band
Band edges	first sample within a MATLAB `find` tolerance	coarse grids miss 1.12, 4, 8, 13 μm and silently return 0
Diode I–V	`fsolve`, warm-started per voltage	no bracket, or a jump too large between voltages
MPP, V_{mpp}	three-point parabolic refinement	peak on a sweep boundary (falls back to the sample)
V_{oc}	interpolated $-J$ zero crossing	sweep does not bracket it (raises)
T_{eq}	interpolated P_{cool} zero crossing	no crossing in range (raises, never returns an endpoint)
V_{mpp} coupling	fixed-point iteration	does not settle (warns, naming the last value)

The band-edge row deserves emphasis because it fails quietly rather than loudly. Both `optical_band_averages` and `pv_band_averages` reproduce the MATLAB convention of taking the first sample within a tolerance of each edge; if the wavelength grid has no sample near an edge, that band average returns 0.0 instead of raising. The default 0.3–30 μm grid with $n = 2000$ satisfies every edge. A coarse grid may not, and a zero in a band-average column should always be read as a possible missing edge before it is read as physics.

The general-purpose `band_average` avoids the tolerance trap by selecting on value, but it is not immune to the wider problem: a band with no samples in range returns 0.0, and a band only partly covered by the grid is averaged over the covered part alone and so reads low. Neither raises. Any band average must therefore be taken together with the wavelength range that produced it.

7 YAML examples

7.1 One file with multiple cases

A top-level `cases` list lets one YAML file define several named runs. The CLI executes them in order:

```
cases:
  - name: normal_TE
    run: {optics: true, thermal: false}
    simulation:
      wavelength: {min: 8.0, max: 13.0, n: 201}
      angles: normal
      polarization: TE
    # geometry, structure, materials, and data follow
  - name: directional_TM
    run: {optics: true, thermal: false}
    simulation:
      wavelength: {min: 8.0, max: 13.0, n: 201}
      angles: specific
      polar_angle_deg: 35.0
      azimuth_angle_deg: 90.0
      polarization: TM
```

YAML anchors may hold shared sections. Validation E uses one such file for nine cases and references digitized text data directly; no helper script is needed.

7.2 Normal and directional emittance

```
run:
  optics: true
  thermal: false
  plots: false
  write_outputs: true

simulation:
  wavelength: {min: 8.0, max: 13.0, n: 300}
  angles: specific
  polar_angle_deg: 35.0
  azimuth_angle_deg: 90.0
  polarization: TM
  s4_modes: 80
```

For normal incidence, set `angles: normal`; the requested TE, TM, or both are still computed explicitly.

7.3 Hemispherical emittance

```
simulation:
  wavelength: {min: 3.0, max: 30.0, n: 1000}
  angles: hemispherical
  polarization: unpolarized
  hemisphere_theta_points: 8
```

```
  hemisphere_azimuth_points: 12
  s4_modes: 100
```

A live S4 thermal case requires `angles: hemispherical`: the energy balance integrates emissivity over the hemisphere, so a single normal-incidence spectrum does not define it. Convergence must be checked by increasing the polar-angle count, azimuth count, and S4 Fourier basis size independently.

That requirement is what makes a live thermal run expensive. The solver cost scales as $n_\lambda \times n_\theta \times n_\phi \times n_{\text{pol}}$: the listing above is 1000 wavelengths over $8 \times 12 = 96$ quadrature directions, plus one zero-weight normal probe, and an unpolarized case evaluates both TE and TM at every one of those points. Changing a convection coefficient or an ambient temperature does not change any of it.

The practical workflow is therefore to run the optics once, write the reduced spectrum, and re-drive the thermal stage from that file with `run.optics_results`, which costs no S4 time. Validations A, B, and C are built this way. Validation C keeps the split explicit: four `optics_*.yaml` cases call S4 and write `data/optics/*.txt`, and two `cell_*.yaml` cases consume those spectra to solve the energy balance.

To chain the two runs without any intermediate step, set `run.optics_export` in the optics case to a fixed path and point `run.optics_results` at that same path in the thermal case. These are the write and read halves of one five-column format (`lambda_um R T emit abs_si`). The export is written even when `run.write_outputs` is false, because it is requested by explicit path. Note that `optics.csv` cannot be reused here: it is comma-separated with a text header, which the resume reader rejects, and it lands in a timestamped folder that a later case cannot name in advance. Case A.1 in `validations/` is the minimal two-command example.

A resumed spectrum retains no directional information, so the atmospheric term is angle-independent whatever the file represents. Be aware that `run.optics_results_angles` does not change this, or any other number: it is recorded in `run.json` and in the export header as a statement of what the stored spectrum is, and `normal` and `hemispherical` produce bit-identical results. Driving the thermal stage from a normal-incidence spectrum remains an angle-independent approximation rather than a hemispherical calculation, and must be reported as such — but it is the provenance of the file, not this key, that makes it so.

7.4 Layer stack and materials

```
geometry:
  source: s4
  shape: cylinder
  photonic_material: sio2
  lattice: {type: hexagonal, x: 10.608811, y: 6.125}
  cylinder: {radius: 1.75, height: 2.25}

structure:
  - {material: sio2, thickness: 500.0}
  - {material: silicon, thickness: 500.0}
  - {material: gold, thickness: 0.08}
  - {material: vacuum, thickness: 0.0, terminal: true}

materials:
  sio2: PalikKitamura_SiO2
  silicon: Akerboom_Si_lossless
```

```
  gold: RII_Olmon_2012_ev_Au
```

Exactly one terminal layer is required, it must be last, and its thickness must be zero because S4 treats it as semi-infinite.

7.5 Thermal case and temperature reduction

```
thermal:
  ambient_temperature: 300.0
  # Calibrated effective total coefficient; the paper states 6.0.
  convection_coefficient: 12.54
  absorbed_solar_power: 808.0
  reference_temperature: 360.0
  cooling_temperature: {min: 260.0, max: 380.0, n: 121}
  equilibrium: auto
```

For a digitized table whose first column is wavelength and later columns are different measured emittances, set `run.optics_results_emittance_column` to the desired zero-based column. Column zero is reserved for wavelength.

8 Outputs and reproducibility

MATLAB-style output compatibility has been removed. New runs write:

- `optics.csv`: selected directional or hemispherical spectrum;
- `optics_directional.csv`: every S4 wavelength, polar angle, azimuth, polarization, angular weight, R, T, A, A_{Si};
- `iv.csv`, `power.csv`, or `cooling_power.csv`;
- `run.json`: full resolved configuration, Git revision, dirty flag, interpreter/platform, S4 binary hash, input paths and SHA-256 hashes, and scalar results;
- figures when `run.plots: true`.

8.1 The `run.json` manifest

Five top-level keys. `resolved_config` is the complete configuration after defaults and validation, so a run can be reproduced without the original YAML. `provenance` carries `python`, `platform`, `git_commit`, `git_dirty`, `inputs` (every resolved input path with its SHA-256), and `s4` (module path and hash, `null` when no live optics ran). `optics` records `angles`, `polarization`, and `n_lambda`.

Ambient and equilibrium appear as pairs so the cost of running hot reads directly off the manifest. The ambient V_{oc} is the highest in the sweep, so a voltage range that legitimately brackets the operating point may not reach it; that case reports `null` rather than 0.0, and does not stop the run, because the equilibrium point is the result.

J_0 and J_{Auger} exist across the whole (T, V) sweep inside `IVResult` — the Auger array alone is 151×100 values — so each is interpolated to T_{eq}, the Auger term additionally at V_{mpp}. The sweeps themselves belong in `iv.csv` and `power.csv`, not in a scalar manifest.

A PV run adds a fifth key, `band_averages_percent`, holding `solar_absorptance_silicon`, `solar_reflectance`, `subgap_reflectance`, `emittance_8_13um` and `emittance_4_30um`: the

`thermal_results` key	Meaning
`equilibrium_temperature_K`	interpolated root of P_{cool}
`temperature_reduction_K`	$T_{\mathrm{ref}} - T_{\mathrm{eq}}$; `null` without a reference
`vmpp_V`	converged fixed-point MPP voltage
`short_circuit_current_A_per_m2`	J_{sc}
`mpp_ambient_W_per_m2`	MPP at T_{amb}
`mpp_equilibrium_W_per_m2`	MPP at T_{eq}
`voc_ambient_V`	V_{oc} at T_{amb}; `null` if outside the sweep
`voc_equilibrium_V`	V_{oc} at T_{eq}
`fill_factor_ambient`	`null` with `voc_ambient_V`
`fill_factor_equilibrium`	fill factor at T_{eq}
`atmospheric_power_W_per_m2`	P_{atm}
`absorbed_solar_power_W_per_m2`	P_{sun}
`temperature_coefficient_perc_per_K`	β_P, negative for silicon
`efficiency_equilibrium`	P_{mpp}/AM1.5 total
`saturation_current_equilibrium_A_per_m2`	$J_0(T_{\mathrm{eq}})$
`auger_current_equilibrium_at_vmpp_A_per_m2`	$J_{\mathrm{Auger}}(T_{\mathrm{eq}}, V_{\mathrm{mpp}})$

weighted averages of Section 6, taken against AM1.5 below the gap and against the blackbody at T_{eq} above it. A PV-free `cooling_curve` run has no operating point to weight them at and omits the key rather than reporting zeros. Read them together with the wavelength grid: a band whose edge the grid misses reports 0.0 rather than raising.

Set `run.write_outputs: false` for tests or programmatic validation. Historical MATLAB files remain only as read-only parity fixtures and under the gitignored `archive/`; they are never emitted by the active package.

9 Validation evidence

9.1 Case detail and reproduction commands

Every case below reproduces the numbers quoted here on the committed inputs. The full statement of sources, digitisation provenance, and per-case limitations lives in `validations/README.md`; this section records what each case establishes and how to re-run it.

A — Silva-Oelker and Jaramillo-Fernandez, *Opt. Express* 30(18), 32965 (2022), Table 1. Five YAML files drive the thermal/PV path from pre-reduced published spectra. They contain no `geometry` block and never call S4, so the paper's hexagonal hemisphere array is implicit in the supplied spectra. The reduced inputs carry no angular TE/TM data, so P_{atm} uses the angle-independent approximation. Computed against published, all five rows: J_{sc} within 3%, P_{mpp} within 3%, T_{eq} within 3 K, V_{oc} within 0.02 V. Reflected power is the weak quantity, reaching about 10%.

```
PYTHONPATH=. python "validations/validation A/run_table1_validation.py"
```

A.1 — hexagonal-cell smoke test. Two plain CLI runs chained by `run.optics_export` and `run.optics_results`. Geometry mirrors Validation A's soda-lime hemisphere row (9 μm diameter on 75 μm flat soda-lime over Si_3N_4/Si/Ag) with an *assumed* close-packed pitch, TE at normal incidence, `s4_modes: 10` and nine slices — none of which is converged. It yields $|R + T + A - 1| \leq 1.1 \times 10^{-16}$ in memory, solar-band $A_{\mathrm{Si}} = 0.823$, window emittance 0.973, $T_{\mathrm{eq}} = 317.09$ K, $J_{\mathrm{sc}} = 365.92$ A/m^2, $V_{\mathrm{oc}} = 0.7264$ V, $P_{\mathrm{mpp}} = 230.61$ W/m^2, fill factor 0.868, efficiency 0.2314. These land near Table 1 but are not a reproduction of it.

```
radcoolpv run "validations/validation A.1/optics_hemisph_sodalime.yaml"
```

Case	Evidence	Conclusion
A	Published reduced spectra; full YAML thermal/PV path	Conditional thermal/PV regression. No live S4 or angular TE/TM validation; reflected-power discrepancies reach approximately 10%.
B	Digitized measured spectra and published curves	Figure 4d tests the cooling balance. Figure 5d is a digitized comparison, not an independent module calculation.
C	Four YAML-defined S4 cases; Zhao et al. grating	Normal unpolarized 8–13 μm emittance is 0.938 versus approximately 0.90; $T_{\mathrm{eq}} = 337.82$ K versus 337.5 K. Solar absorptance is prescribed and normal optics are used as an angle-independent thermal approximation.
E	One YAML; S4 spectra and digitized Akerboom et al. Figures 3a, 5a, 5b	The 7.5–16 μm S4 RMSE is 0.025–0.030, but the silica cases have 2–16 μm RMSE 0.174 because the finite stack differs from the paper's semi-infinite-silica absorption convention. The paper's $h_c = 6.0$ does not reproduce Figure 5b. For the paper's zero-emitter case it gives 434.67 K, whereas the paper reports 366.5 K; that value requires $h = 12.15\ \mathrm{W\,m^{-2}\,K^{-1}}$. Separate YAML cases retain the failed paper input and a joint fit $h = 12.54\ \mathrm{W\,m^{-2}\,K^{-1}}$, which gives 359.7/340.1/337.5 K versus 360.0/339.0/336.0 K. The latter is calibrated reproduction, not convection validation.
A.1	Live S4, TE at normal incidence, hexagonal cell	**Not a validation.** A configuration smoke test with no published comparison: it checks that a hexagonal cell with a discretised semisphere solves and drives the PV stage. Its array pitch is assumed, not taken from a paper.

```
radcoolpv run "validations/validation A.1/pv_hemisph_sodalime.yaml"
```

B — Le et al., *ACS Photonics* (2026), Figures 4d and 5d. Figure 4d runs `cooling_curve` from digitized film spectra at 800 $\mathrm{W\,m^{-2}}$ and $h_c = 9\ \mathrm{W\,m^{-2}\,K^{-1}}$. Figure 5d is a digitized curve comparison only: the paper does not supply the raw commercial-module optics an independent calculation would need.

C — Zhao et al., *Renewable Energy* 191 (2022) 662–668. The only case exercising the 1-D `grating` shape ($p = 7$ μm, duty 0.2, depth 10 μm). The grating fills silica's 9 μm reststrahlen dip, lifting the window-averaged emittance from 0.767 (planar) to 0.938 against the paper's ≈ 0.90; the coupled model gives $T_{\mathrm{eq}} = 337.82$ K against 337.5 K. Both normal-incidence polarizations are evaluated and the 20–80-mode window average varies by less than 0.001. Conditional on a prescribed solar absorptance $\alpha = 0.95$ and on normal optics used as an angle-independent thermal input.

E — Akerboom et al., *ACS Photonics* 9 (2022) 3831–3840. One YAML defines nine named cases. Optics agree in the radiative-cooling band (7.5–16 μm RMSE 0.025–0.030) but not over the full 2–16 μm range for the silica stacks (RMSE 0.174), because the finite fabricated stack differs

from the paper's semi-infinite silica absorption convention. The thermal half fails on the paper's own terms and the repository keeps that failure visible rather than tuning it away.

```
radcoolpv run "validations/validation E/validation.yaml"
```

9.2 Standing limitations

Validation E has not received a fresh Fourier-mode convergence sweep. Its 60-mode patterned result must not be labelled a converged reference solution.

10 Required verification for new cases

For every new publication or device case:

1. place all geometry, materials, directions, polarization, wavelength, and numerical settings in YAML;
2. record source data and reject unavailable parameters rather than inventing them;
3. verify $R + T + A$, passivity, material wavelength coverage, and layer-resolved absorption;
4. demonstrate wavelength, Fourier-mode, polar-angle, and azimuth convergence as applicable;
5. distinguish unit tests, MATLAB parity, independent optical validation, literature reproduction, digitized comparison, and calibrated fit;
6. compare published and calculated values with absolute or relative errors and retain unresolved failures.

References

[1] V. Liu and S. Fan, "S4: A free electromagnetic solver for layered periodic structures," https://web.stanford.edu/group/fan/S4/.

[2] B. Zhao et al., "Radiative cooling of solar cells with micro-grating photonic cooler," *Renewable Energy* 191, 662–668 (2022), https://doi.org/10.1016/j.renene.2022.04.063.

[3] E. Akerboom et al., "Passive Radiative Cooling of Silicon Solar Modules with Photonic Silica Microcylinders," *ACS Photonics* 9, 3831–3840 (2022), https://doi.org/10.1021/acsphotonics.2c01389.

[4] R. L. Olmon et al., "Optical dielectric function of gold," *Physical Review B* 86, 235147 (2012), https://doi.org/10.1103/PhysRevB.86.235147.

[5] M. N. Polyanskiy, "Refractiveindex.info database of optical constants," *Scientific Data* 11, 94 (2024), https://doi.org/10.1038/s41597-023-02898-2.